PERSONAL MACHINES · TIER 1

End-to-End Product Workflow (v3.00) — User Guide

Last updated: August 11, 2026

See your product before you build. Scope deliberately. Scale on a plan that holds up.

From idea to shipped product, end-to-end.

WORK IN PROGRESS

This workflow is still evolving. We welcome your feedback and appreciate your patience as we continue to iterate and improve.

A note on tiers. Steps 1–8 are the same for every tier. The workflow only branches at **Step 9** (plan the build), so you don't need to nail down your tier before getting started — just confirm it before kicking off `/plan`.

A note on screenshots. Claude Design's UI gets updated regularly, so the screenshots in this guide may look slightly different from what you see. The flow stays the same — look for the same buttons and options even if their styling has changed.

What's new in v3.00 — if you've used an earlier version, here's what changed and where to look:

- **Step 4 (Create a blueprint)** — added a **Don't use real data** callout. Anything you type into requirements flows into the HANDOFF brief and blueprint, then into Claude Design, where stakeholders can see it and Claude Design retains a copy. Use fake data — never real client info, PII, financials, or confidential matter details.
- **Step 5 (Build the prototype)** — removed the substeps that had you invoke the `/Interactive prototype` skill and select the **UI mockups** template. Both are handled automatically now.

Quick steps

A note on shipping. `/ship` isn't only the final step — you can run it at any point in the workflow to save the current step's work to the `dev` branch. See **Step 12**.

1. **Set up a new application** — in Windows PowerShell or Terminal, install the project template and create your app folder. Repeat for each new project.
2. **Launch Claude Desktop**, go to the **Code tab**, and change the working folder to the one you created in Step 1.
3. **Describe your product** to **`/requirements-gathering`** in Claude Code. It asks follow-up questions and writes up the requirements for the later steps.
4. **Run `/design-foundation` in Claude Code**. It reads your requirements, proposes which screens to prototype, and generates a **visual sitemap** so you can review the selection as a flowchart before you sign off.
5. **Build screens one at a time** in Claude Design. *(First time? Set up the McDermott Design System first — see **Setup**, below.)* Start a new prototype by dropping in the HANDOFF brief with the McDermott Design System selected, and steer through screens in flow order.
6. **Share for feedback** *(optional)* — export the prototype from Claude Design and refine based on what you hear back.
7. **Download your prototype as a .zip from Claude Design.**
8. **Run `/design-code-handoff` in Claude Code** and drag or upload the `.zip` into the chat. Claude Code files the archive and reconciles the plan to match your prototype — **automatically, with no sign-off**.
9. **Run `/plan` in Claude Code** to start the build pipeline. Stay available to review what gets built.
10. **Run `/build` in Claude Code** to implement the build plan. Stay available to review what gets built.
11. **Run `/review` in Claude Code** to check the code — tests, guardrails, layered code review, and a security audit. `/ship` won't run without a clean review.
12. **Run `/ship` in Claude Code** to save your work to the `dev` branch — for code it needs a clean `/review`; for docs it scans for sensitive info. Run it after any step, not only at the end when you deploy the finished product.

13. **Get a second opinion** — open a new Claude Code session and run `/second-opinion` to review and check the app you've built.
14. **Test the app locally** — run `/local-testing` to launch the frontend, API, and database locally.

💼 Step 1 — One-time setup for a new application

Windows PowerShell or Terminal

YOUR PART — *Install the project template and create your app folder.*

PREREQUISITE

Before starting Step 1, complete the machine configuration guide for your OS — **Windows** or **Mac** (PDF, one-time setup per machine). It installs the tools you'll use below and creates the GitHub access token you'll need.

1. Install the latest solutions template

Run this before every new project so you get the latest template. In Windows PowerShell or Terminal, copy-paste and run one command at a time:

```
npm cache clean --force
npm install -g @alan-eicker/northstar-cli
```

Note: If you get an execution policy error, run this command first:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

2. Create your APP folder

Navigate to `C:\Users\[user_name]\` in File Explorer and create a new folder named `claude_apps` if it doesn't exist already. Replace `[user_name]` with your username on the machine — check `C:\Users\` to identify this.

Run these two commands in Windows PowerShell or Terminal. Replace `[user_name]` and `[app_name]` with your actual values:

```
cd C:\Users\[user_name]\claude_apps
nstar create [app_name]
```

You'll get prompted with a few options:

- **GitHub account type?** Enter `1` (Personal).

- **GitHub access token?** Paste the token you created during the machine configuration guide (linked at the top of this step).

> **Note:** The token won't appear in the prompt for security reasons — it will look empty. Just paste and hit Enter.

After this step, a folder named `[app_name]` should exist at `C:\Users\[user_name]\claude_apps\`. Confirm manually using File Explorer.

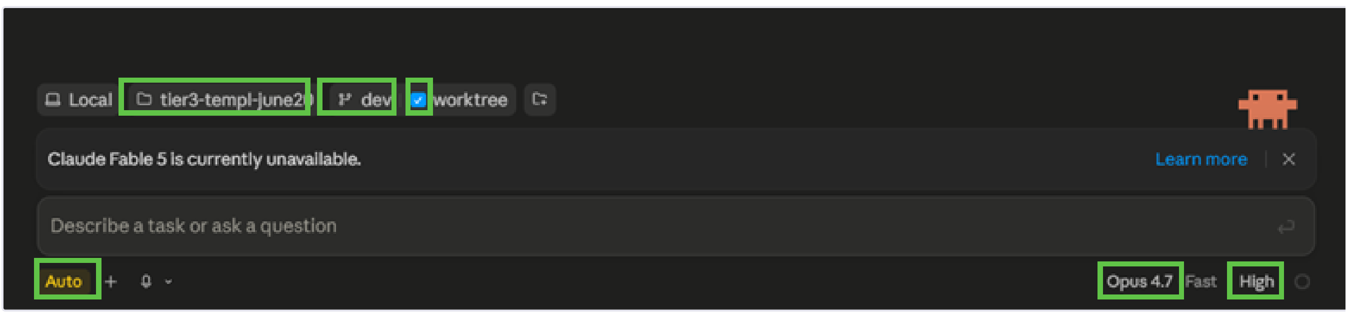
🖵 Step 2 — Open your project in Claude Code

Claude Desktop

> **YOUR PART —** *Launch Claude Desktop, point Claude Code at your app folder, and confirm your settings.*

Launch **Claude Desktop** and go to the **Code tab**, then make sure these are selected:

1. Working folder: the one you created in **Step 1**
2. Branch: **dev** (with **worktree** checked)
3. Mode: **auto**
4. Model: **Opus 4.7**, **high** effort



Confirm these settings before you run anything: dev branch with worktree, Opus 4.7 at high effort, and auto mode.

Step 3 — Gather your requirements

Claude Code · Opus 4.7

> **YOUR PART —** *Run `/requirements-gathering` to describe your product.*

Within your project in Claude Code, run **`/requirements-gathering`**. Describe your product to Claude — it'll ask follow-up questions and write up your requirements.

> **Tip:** If Claude asks something you can't answer yet, pause and think. Guesses harden into product decisions.

Step 4 — Create a blueprint

Claude Code · Opus 4.7

YOUR PART — *Review the sitemap and sign off on the proposed screens.*

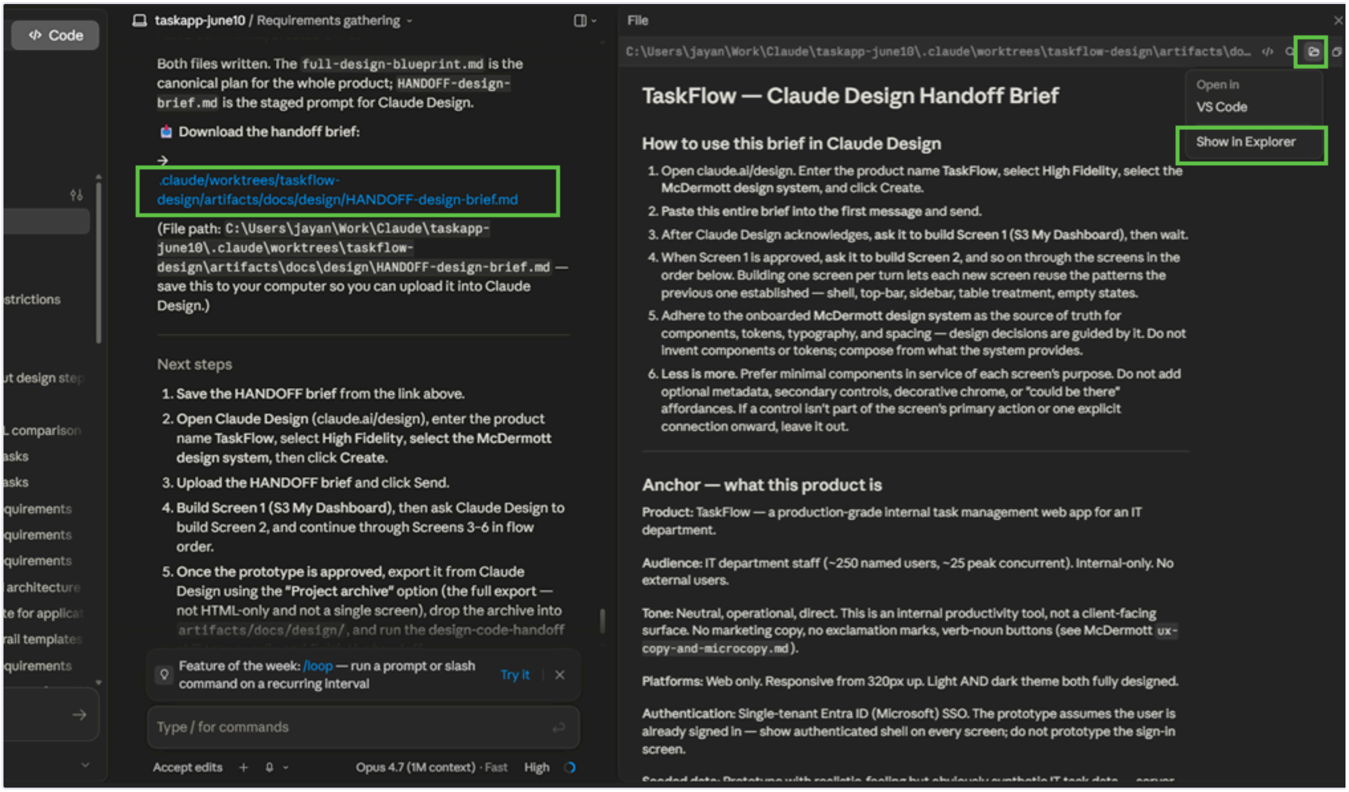
DON'T USE REAL DATA IN YOUR PROTOTYPES

Use fake data for your prototypes — always. Anything you type into requirements flows into the HANDOFF brief and blueprint, then into Claude Design, where stakeholders reviewing the prototype can see it and Claude Design retains a copy. **Our guardrails are not enforced in Claude Design.**

Fake means fake: made-up client names, sample matter numbers, placeholder dollar amounts. Never real client info, PII, financials, or confidential matter details.

Your data is your responsibility.

1. Run `/design-foundation`. It reads your requirements, proposes a screen selection, and generates a **visual sitemap**.
2. **Open the sitemap** to review which screens are in the prototype, which are saved for `/build`, and which Claude suggested adding.
3. **Sign off on the selection** — reply with your approval, or tell Claude what to change and re-review the updated sitemap. Claude waits here and won't continue until you approve.
4. **Open the HANDOFF brief.** Claude writes the design blueprint and the **HANDOFF brief** into `artifacts/docs/design/` in your active branch — no download needed. Click the HANDOFF brief's link in Claude Code to open it, or find it in **Finder** (Mac) or **File Explorer** (Windows). You'll upload this file into Claude Design in Step 5.



Click the HANDOFF-design-brief.md link in Claude Code to open it from your project folder.

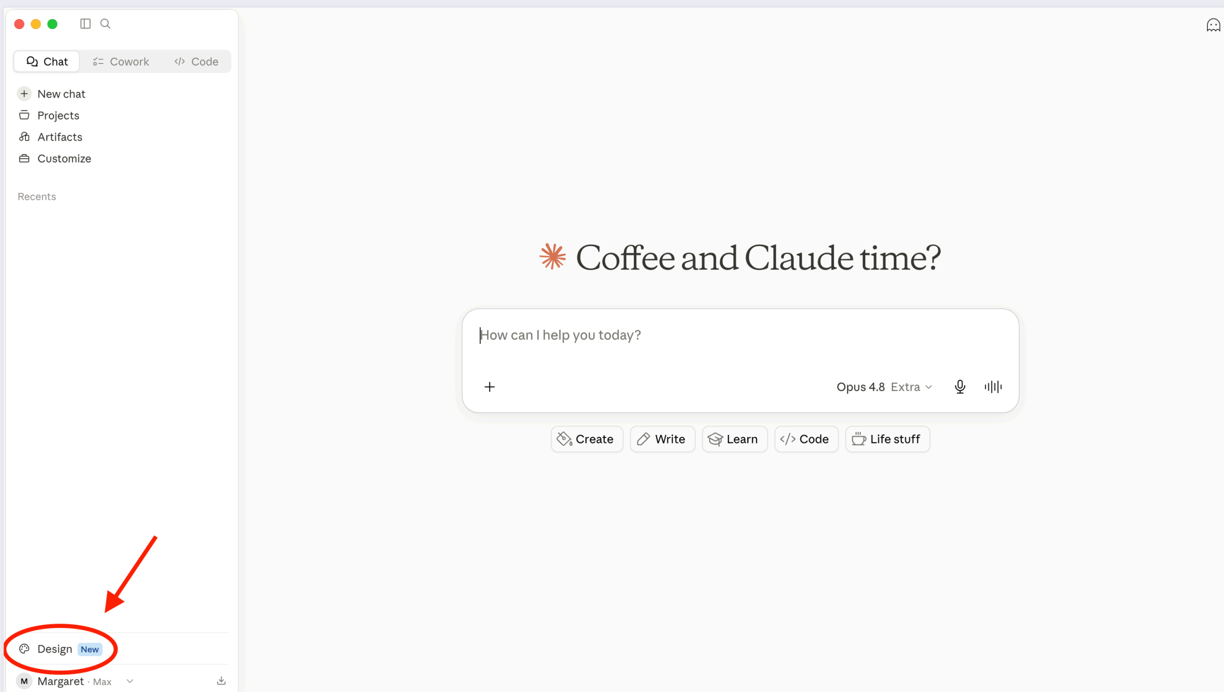
⚙ Setup — design system in Claude Design *(one-time, if needed)*

Claude Design

> **YOUR PART —** *Set up the McDermott Design System in Claude Design (if it's not already there).*

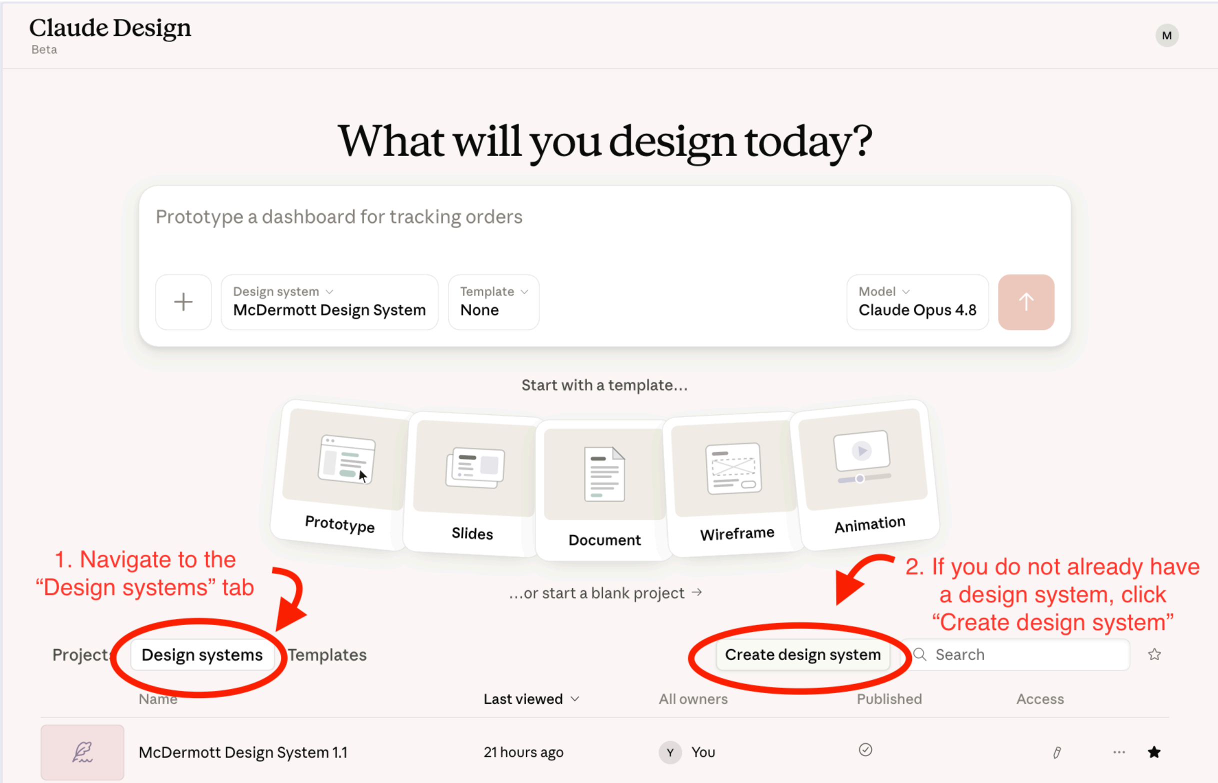
The McDermott Design System is the foundation Claude Design uses to build your prototype. The more complete and polished it is, the better the prototype will look — and the closer your final build will be to it.

- Open **Claude Design** in the Desktop App under Chat or Cowork or in your browser at claude.ai/design.



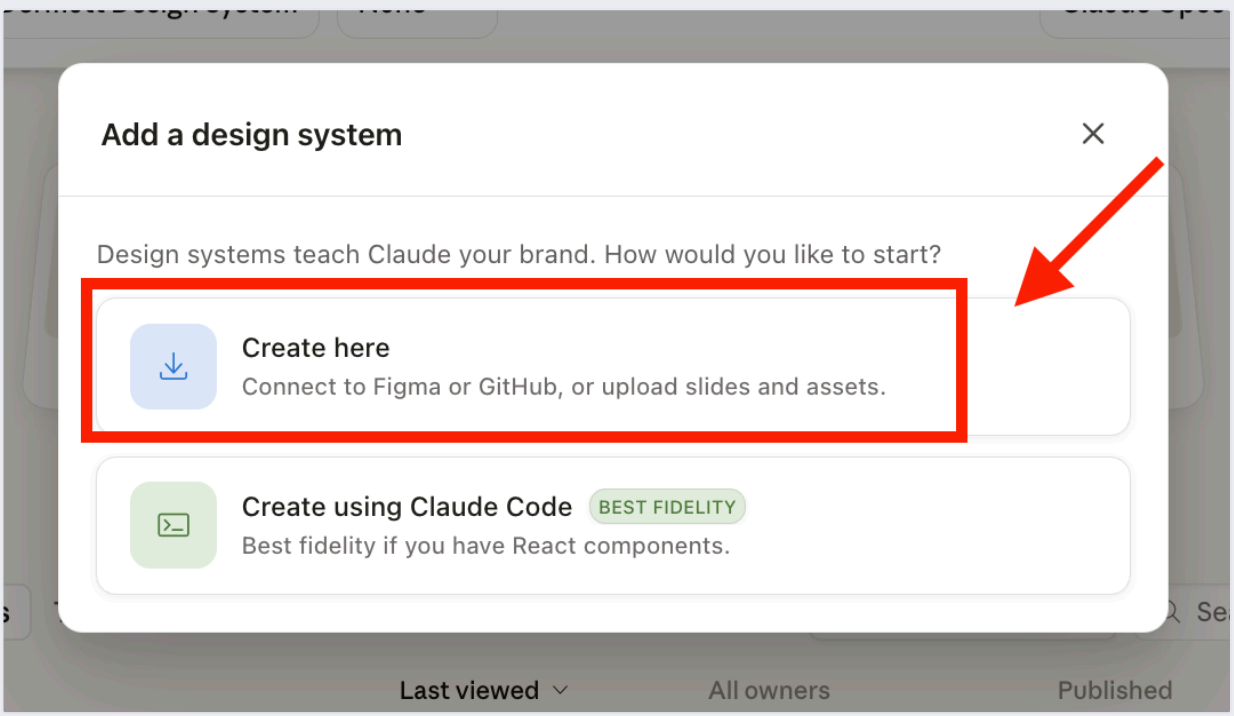
Find Claude Design in the Desktop App under Chat.

- Check your design systems list. If you already have a McDermott Design System, you can skip the setup process and go to **Step 5 — Build the prototype**. If not, click **"Create design system"** to begin setup.



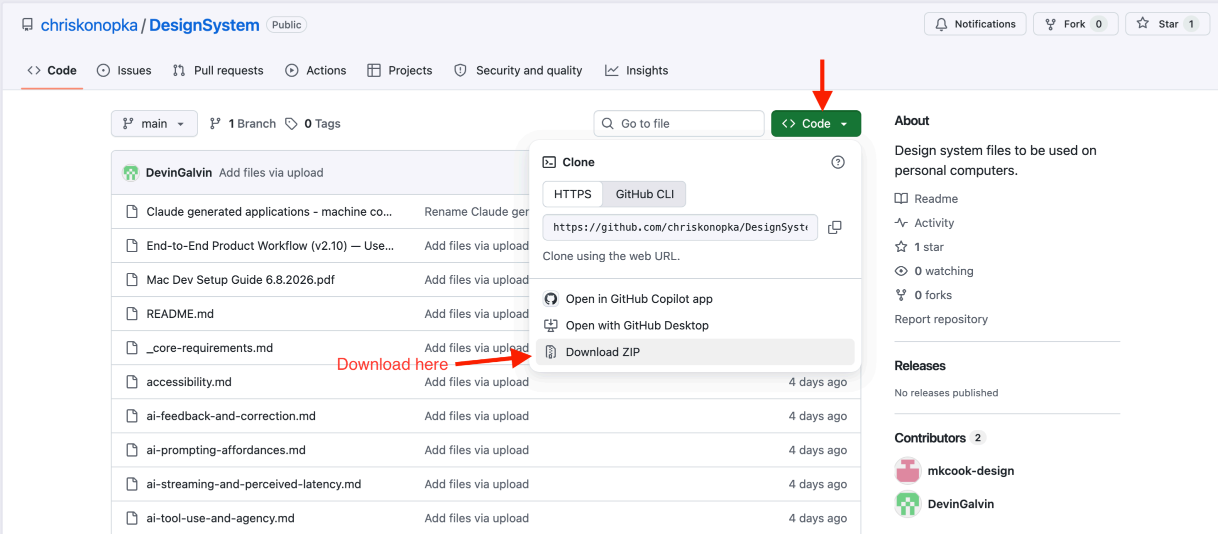
Check your design systems list.

- When prompted, select **"Create here"**.



When prompted, select "Create here".

- Download the latest **McDermott Design System file package** from GitHub.



GitHub download: navigate the McDermott Design System repo and download the files.

- **Unzip the downloaded folder**, then upload the files into Claude Design through the **"Link code on your personal computer"** option to set up the design system.

> **Tip:** Some users have had trouble uploading the files directly from their Downloads folder. If you run into issues, move the unzipped folder to your desktop (or another location on your machine) first.

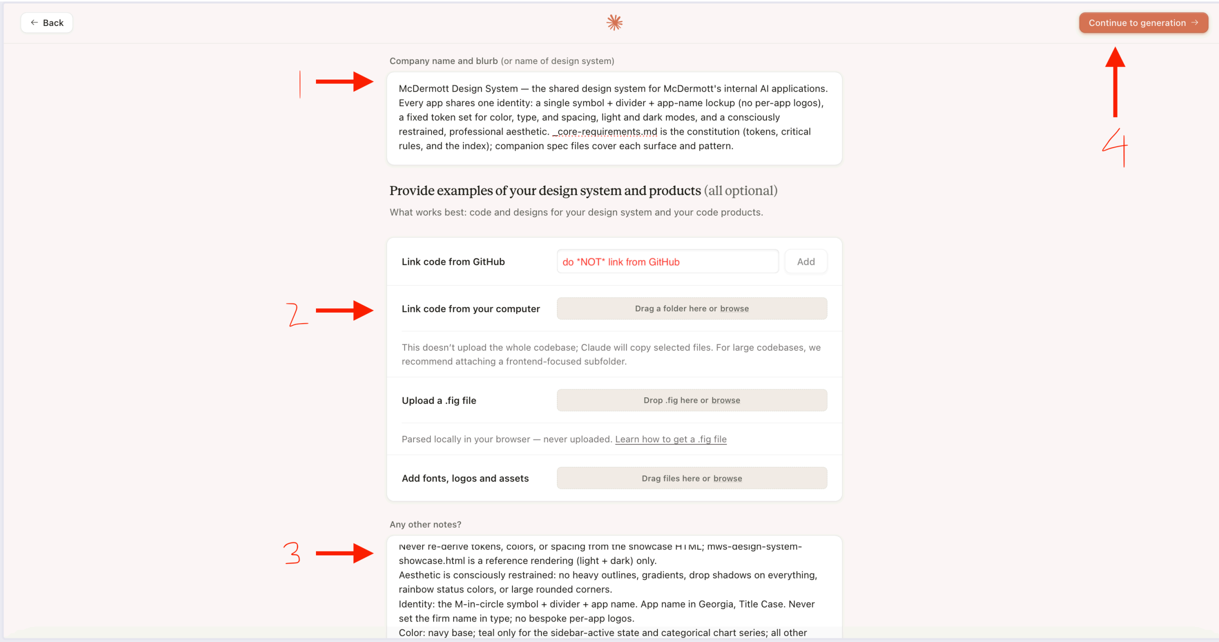
In the "Company name and blurb (or name of design system)" field, paste:

> McDermott Design System — Enterprise-grade navy + pale + accent visual system. Saturated colors are targeted spice, not default tools.

In the "Any other notes?" field, paste:

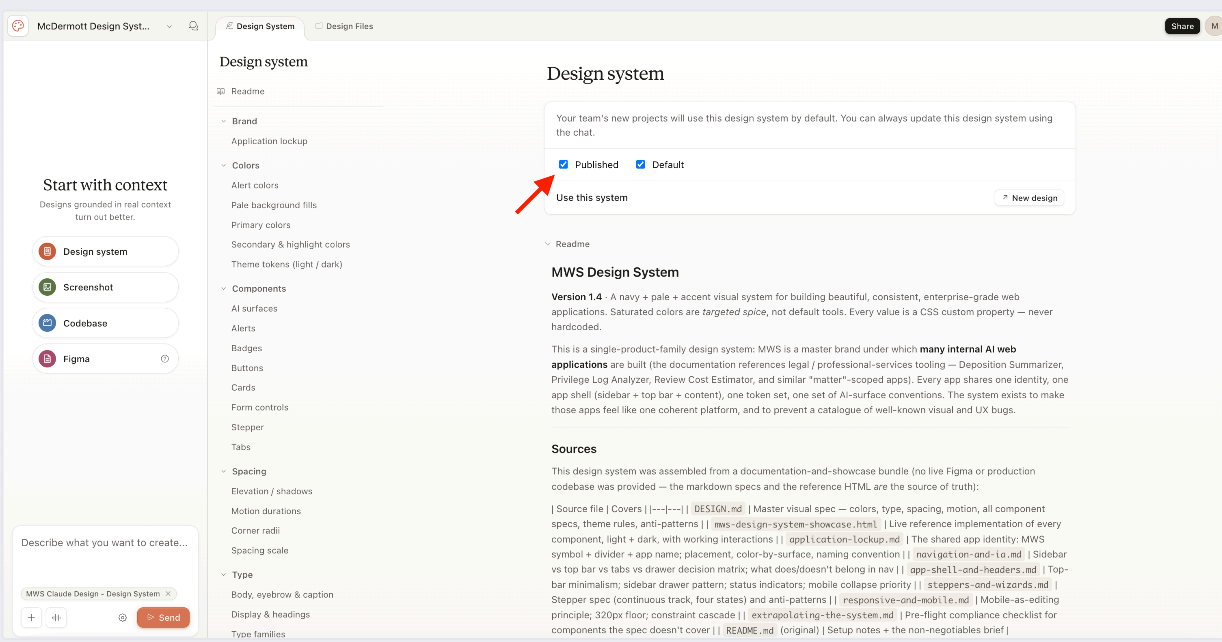
McDermott Design System non-negotiables:

1. Stepper: circles on a continuous 1px track. NEVER bordered rectangles around each step. Four states (not-started / current / completed / error), all same size, same baseline.
2. Match the nav to the app's depth; never default to a sidebar. Deep, multi-section apps → side nav. Shallow apps (≤~7 sections) → nav links live inline in the top bar, no sidebar. Single-purpose apps (one form / wizard / focused tool) → no global nav at all; the top bar is just chrome (page title + primary action + account). Never add a sidebar "for consistency." When a sidebar does exist, it's app navigation only (e.g. Summary, Reference, Admin) — NEVER mirror the canvas stepper in it.
3. Top bar at narrow viewports: hamburger (only if the app has global nav to collapse) + current page name + ONE primary action + theme/account icons. Never wrap text word-by-word. Move computed totals out of the top bar — to the sidebar matter card, or a sticky summary in the content area.
4. Below 1024px: when there's a sidebar, it slides in as a drawer from the left with scrim. Never stacks above content, never disappears entirely.
5. Canvas content is centered, never pinned left. A width-capped region (reading/forms, max-width ~1200px) gets margin-inline: auto so it sits centered in the canvas with balanced gutters — not flush-left with empty space on the right. Data-dense surfaces (tables, dashboards) go full-width with symmetric gutters instead. "Centered" = the content column is centered, not text-aligned center.
6. Buttons never wrap (white-space: nowrap). Card content reflows like a standalone form — segmented controls wrap or become selects on mobile.
7. Navy + pale + accent. var(--color-teal) direct ONLY for the dark sidebar's active state and categorical chart series. Everything else interactive uses var(--accent-interactive) — blue in light mode, teal in dark.
8. Text on pale backgrounds and alert fills is ALWAYS navy, never var(--text-primary).
9. Border radius is 2px or 999px. Nothing in between.



Set up the McDermott Design System in Claude Design by uploading the files.

- Wait for the design system to finish setting up.
- **Publish the design system** to make it available for use in your projects. Without publishing, the system won't show up when you go to build a prototype.



Publish the design system in Claude Design to make it available for use.

- **Verify all components are loaded.** After publishing, paste this prompt into Claude Design to confirm the upload was complete:

> "List all components, tokens, and patterns from the McDermott Design System you can use, and flag anything missing from the upload."

This catches any gaps before you start building prototypes in Step 5.

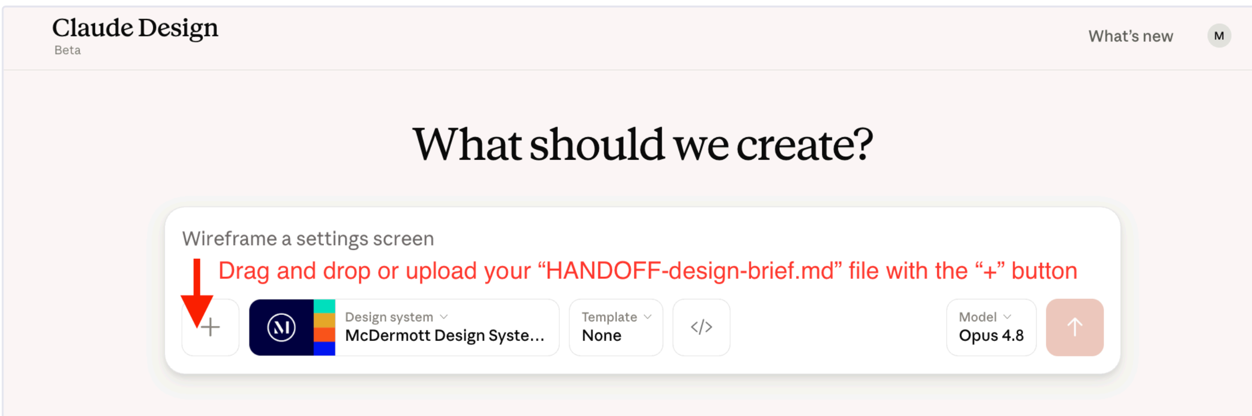
Tip: A thorough McDermott Design System is the biggest factor in a clean prototype. A thin system (just a logo and a color) produces improvised-looking output. Time invested here pays off at every later step.

🎨 Step 5 — Build the prototype

Claude Design

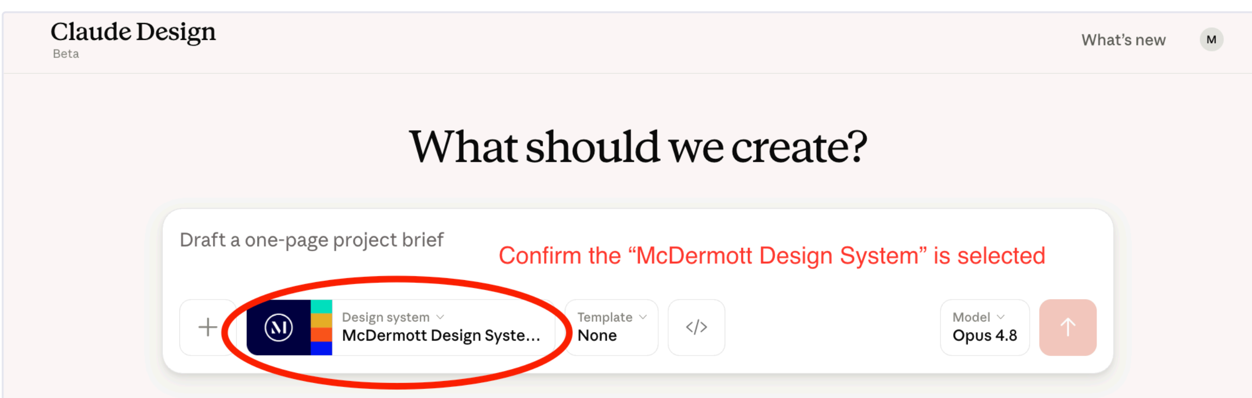
> **YOUR PART —** *Build screens one at a time in Claude Design.*

1. In **Claude Design**, create your design file:
   a. Drag and drop or upload your `HANDOFF-design-brief.md` file (from `artifacts/docs/design/`, written in Step 4) with the **"+"** button.



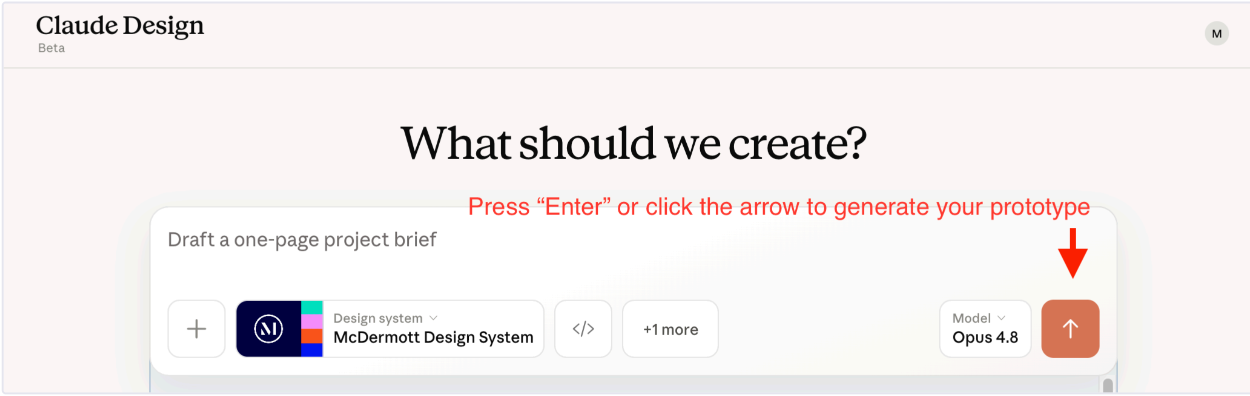
Drag and drop or upload your "HANDOFF-design-brief.md" file with the "+" button.

   b. Confirm the **McDermott Design System** is selected.



Confirm the "McDermott Design System" is selected.

   c. Click the orange button or press **"Enter"** on your keyboard to generate your prototype.



Click the orange button or press "Enter" to generate your prototype.

2. **Build screen 1 first.** When you're happy with it, **prompt Claude to build the next one** by saying something like *"now build screen 2"*. Continue prompting for each new screen — Claude builds one at a time and won't move forward on its own.
3. Refine any screen by commenting on it, editing text directly, or using the tweaks panel.

> ⓘ **IMPORTANT — USE THE FIRM-LICENSED FONTS AND ICONS**
>
> **The prototype must use the typefaces McDermott is licensed for** (**Arial** and **Georgia**), and **Phosphor** (outline only) for icons. They ship with the McDermott Design System, so you get them by default. Don't ask Claude Design to swap in a different typeface or icon set while building or refining screens.

> **Tip:** It's fine to build only some of the screens, not all of them. If you stop early — because you ran out of time, decided a screen isn't worth prototyping, or just want to ship what you have — design-code-handoff in Step 8 catches the unbuilt screens automatically. By default it keeps each one as a **"save for `/build` "** screen (built later from the blueprint), so nothing is lost — it only drops a screen if you explicitly asked to delete it while prototyping. You don't decide screen by screen anymore; it's handled silently and listed in the summary.

🗨 Step 6 — (Optional) Share for review

Claude Design

> **YOUR PART —** *Export the prototype and collect feedback.*

Export the prototype as a shareable URL, HTML, PDF, or PPTX. Collect feedback. Refine in Claude Design and re-export until you're happy.

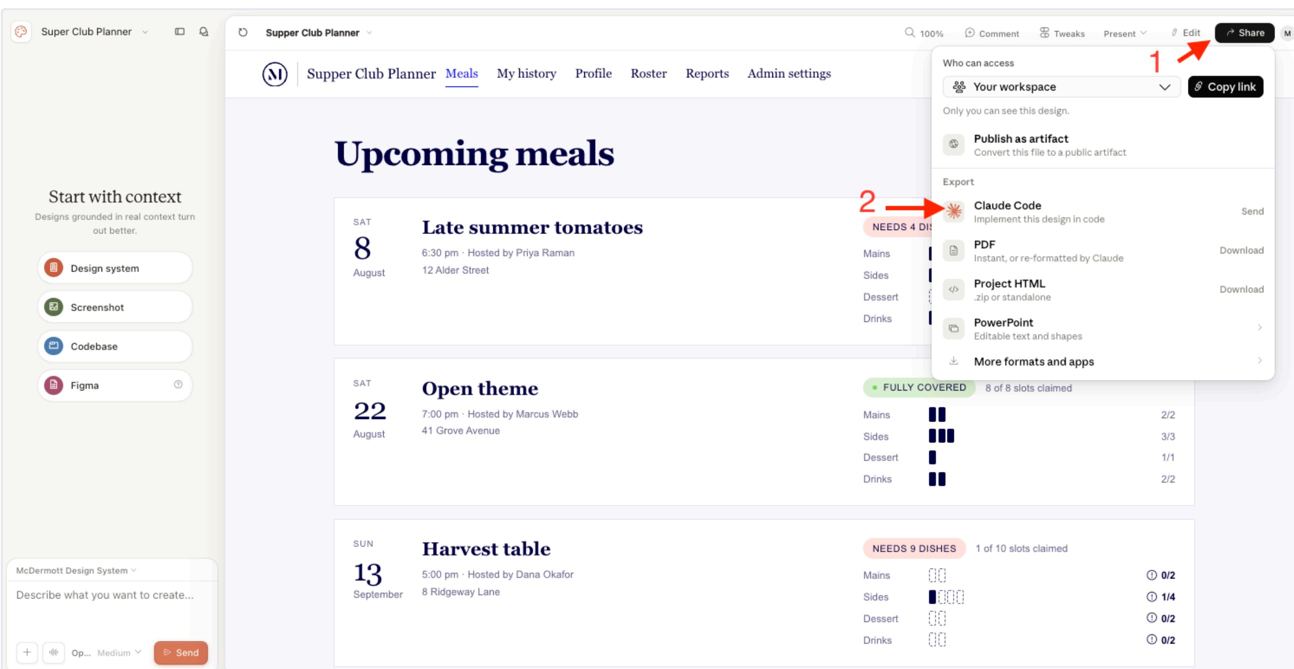
Step 7 — Export the project archive

Claude Design + your computer

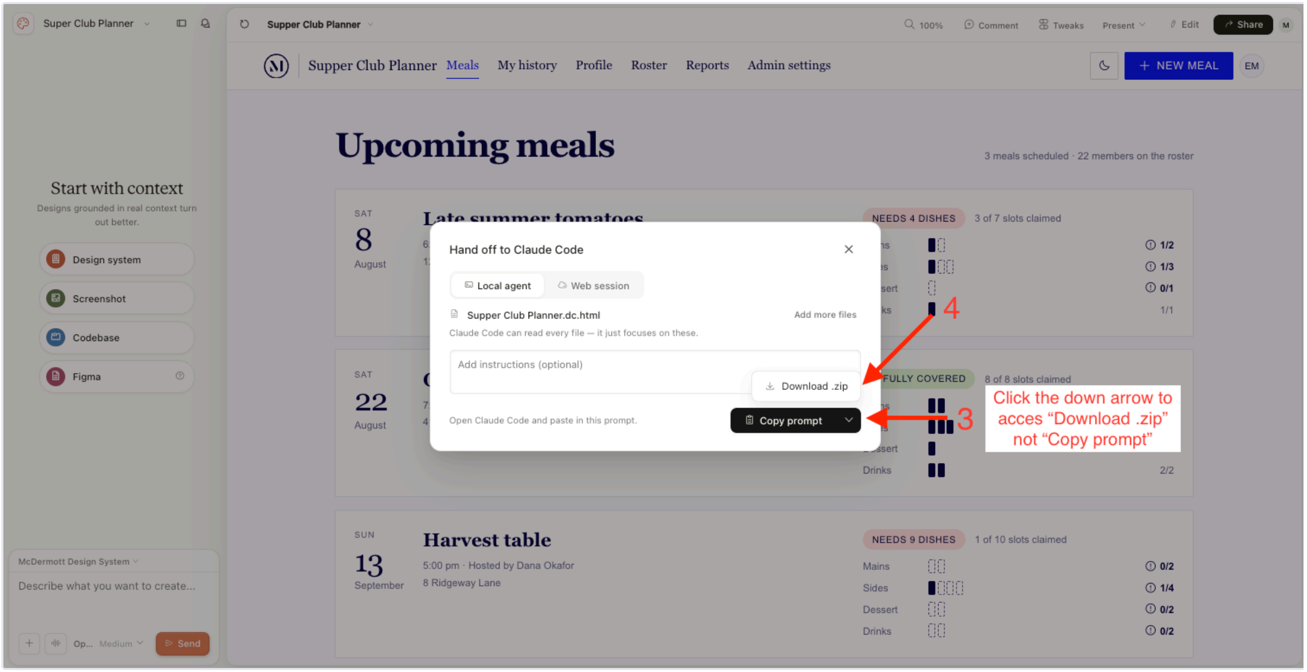
> **YOUR PART —** *Export the prototype .zip.*

In **Claude Design**, export the approved prototype as a project archive (a `.zip` of the full package — not the HTML-only export or a single-screen export). Two ways to do this — **Option A is the preferred path**.

- ☆ **Option A (preferred) — Share with Claude Code:**
  1. Click the **Share button** in the top-right corner.
  2. Under **Export**, click **"Send"** next to **Claude Code**.
  3. In the modal that opens, check **"Download zip instead"**.
  4. Click **"Download .zip"**.



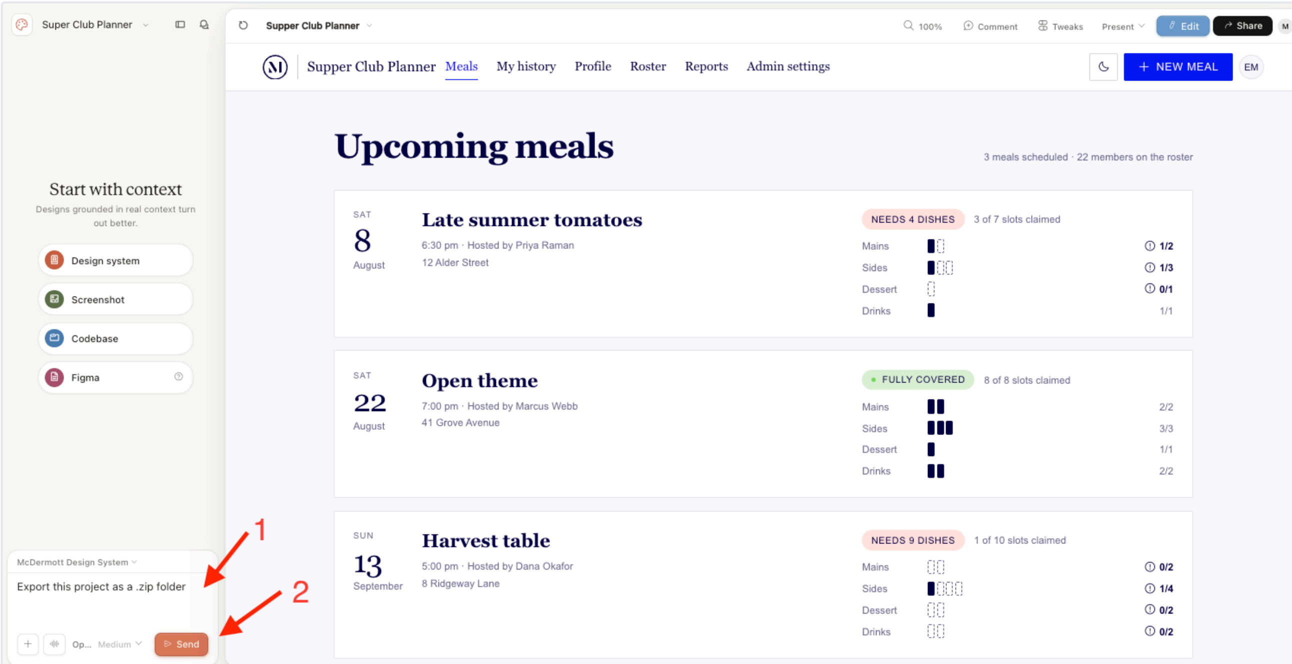
Steps 1-2: Click the Share button, then click Send next to Claude Code under Export.



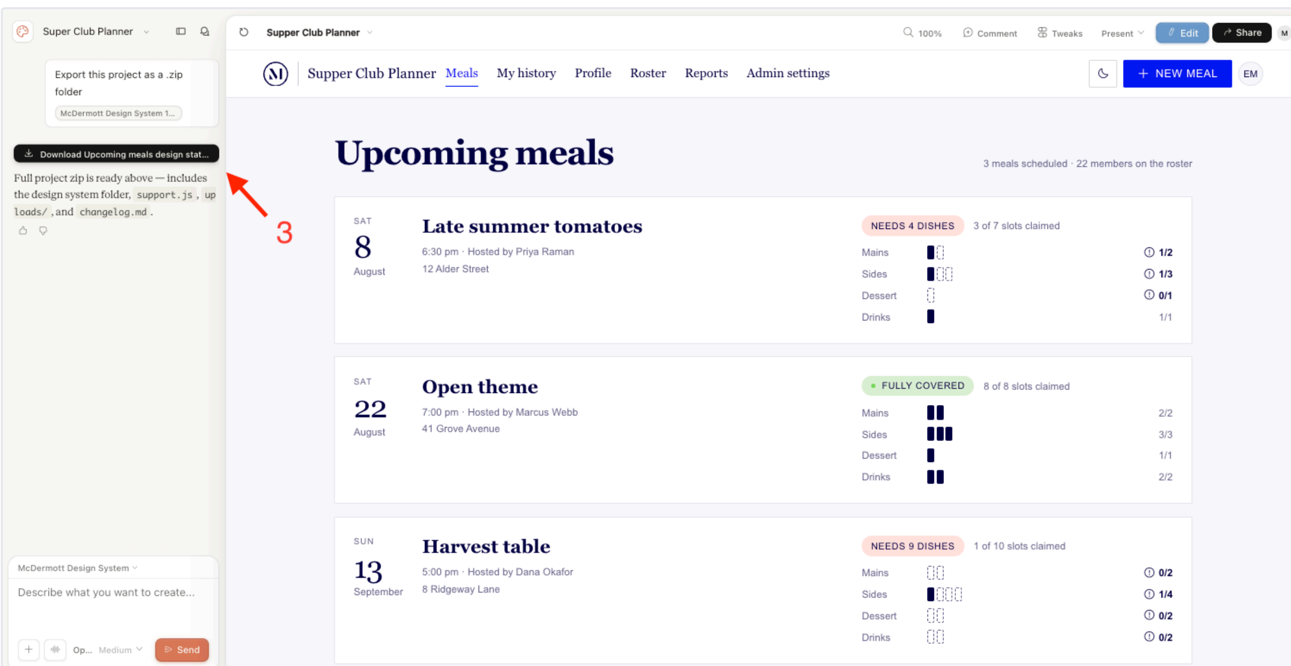
Steps 3-4: Check "Download zip instead," then click "Download .zip."

- **Option B — Ask in chat:**
  1. In the **chat panel** on the left, type *"I would like to export this **project** as a .zip file"*.
  2. Click **Send**.
  3. Download the zip from Claude Design's response.

> ⓘ **CHECK THE RESPONSE BEFORE DOWNLOADING**
>
> Make sure the message with the download says the **"full project zip is ready"** — that confirms you're getting the complete project archive and not a partial or single-screen export. If it doesn't, re-ask in chat.



Steps 1-2: Type your request in the chat panel, then click Send.



Step 3: Download the zip from Claude Design's response.

> **Tip:** Don't unzip or rearrange the project archive. Leave it exactly as Claude Design created it — the next step's skill expects that arrangement and handles the rest.

✓ Step 8 — Reconcile the plan with the prototype

Claude Code · Opus 4.7

YOUR PART — *Hand the prototype .zip to Claude Code.*

In **Claude Code**, run **`/design-code-handoff`** from your active branch. When the skill asks for the prototype archive from Step 7, **drag or upload the `.zip` into the Claude Code chat**.

Claude Code takes it from there — it files the archive into your project and reconciles the plan to match your prototype, automatically. **What gets built is your prototype, plus the screens you saved for `/build`.** Move on to Step 9 when it's done.

> ⓘ **THE SKILL MOVES THE FILE — IT DOESN'T COPY IT**
>
> After `/design-code-handoff` files the `.zip` into your project, it's no longer in your Downloads folder. If you want to keep a copy in Downloads, **duplicate the file first** (⌘D on Mac, Ctrl+C then Ctrl+V on Windows) before uploading.

> ⓘ **DON'T EDIT ANYTHING AFTER THE HANDOFF**
>
> Don't hand-edit `solution-requirements.md` or `full-design-blueprint.md`, and don't change the prototype in Claude Design either — unless you re-run `/design-code-handoff` afterward. Step 8 has already reconciled everything to your prototype, and `/plan` reads the files as-is; any later edits silently desync the build. Do all your shaping in Steps 3–8.

> **Tip:** If uploading or dragging into chat does not work, try placing the `.zip` in your **active branch's** `artifacts/docs/design/` folder yourself (the same folder you opened the HANDOFF brief from in Step 4) and reply **"ready"** when the skill asks. Not sure which folder that is? Ask Claude in the chat where it wants you to put the `.zip` file.

🚀 Step 9 — Plan the build

Claude Code · Opus 4.7

> **YOUR PART —** *Run `/plan` and answer scoping questions as they come up.*

In **Claude Code**, run **`/plan`** to start the build pipeline. It reads everything in place — your reconciled blueprint, requirements doc, and prototype export folder — and produces the build plan that drives the rest of the implementation.

> **Note:** Claude may pause with clarifying questions about scope, tier, or edge cases — answer them promptly; the plan's quality depends on it. When it finishes, skim the plan before running `/build` — catching scope issues here is much cheaper than fixing them mid-build.

🔨 Step 10 — Build the product

Claude Code · Sonnet 5

> **YOUR PART —** *Switch to Sonnet 5, then run `/build`.*

> **Switch to Sonnet 5 for this step.** In Claude Code's model picker, change from Opus 4.7 to **Sonnet 5** before running `/build`. Sonnet 5 is faster and better-suited for the implementation work in `/build` and `/ship`.

In **Claude Code**, run **`/build`** to implement the build plan from Step 9. It writes the code for your product, screen by screen — building the prototyped screens to match the prototype, and the remaining "save for `/build`" screens from the blueprint, styled to match.

🔍 Step 11 — Review the code

Claude Code · Sonnet 5

YOUR PART — *Run `/review`, then clear anything it flags.*

Stay on Sonnet 5 for this step. If you're continuing from Step 10 you're already set. Otherwise, switch to **Sonnet 5** in Claude Code's model picker before running `/review`.

In **Claude Code**, run **`/review`**. It checks your **code changes only** and produces a clean review record — which **Step 12's `/ship` requires** before it will ship code.

`/review` runs:

- The test suite
- Guardrails
- A layered code review
- An OWASP security audit

Docs-only change? Skip this step. `/review` covers code only. If this round changed only documents (requirements, blueprint, etc.), go straight to Step 12 — `/ship` scans documents for sensitive info on its own.

Note: Re-run `/review` if you touch the code after a clean review — the record is tied to the exact code that was reviewed.

When to push back

This is a long process, and Claude will sometimes look for ways to take shortcuts or skip work — declaring something impossible, proposing a lighter alternative, or handing the decision back to you. **Always question Claude, especially when it asks you to decide.** A prompt for approval can be a place where Claude would prefer you signed off on less work; make sure you understand what's being traded away before you agree.

This applies throughout the workflow — not just `/review`. When Claude says it can't do something, or offers a shortcut, or asks you to choose between a full approach and a quicker one, push back with these prompts:

- **"Explain why you can't do this."** Get the specific blocker in plain terms, not a generic "it's not possible."
- **"Do more research before deciding."** Ask Claude to dig into the code, docs, or dependencies before concluding it's blocked.
- **"What are the actual problems here?"** Get a concrete list of what's in the way — often the real obstacle is smaller (or different or nonexistent) than the first refusal suggests.
- **"Proceed anyway — don't take the shortcut."** When Claude says something is too time-consuming and offers a lighter alternative, don't always accept the shortcut. Ask it why it makes more sense than proceeding with the full approach.

Most "I can't" responses turn into "here's how" after one or two rounds of this.

Step 12 — Ship the product

Claude Code · Sonnet 5

> **YOUR PART —** *Stay on Sonnet 5, then run `/ship`.*

> **Stay on Sonnet 5 for this step.** If you're continuing from Step 11 you're already set. Otherwise, switch to **Sonnet 5** in Claude Code's model picker before running `/ship`.

In **Claude Code**, run **`/ship`**. It verifies the right check has passed, then **commits your work, merges it into `dev`, and pushes.** `/ship` never runs `/review` itself — it only checks that you did.

Before `/ship` lands anything, it checks:

Shipping…	`/ship` requires	If the check fails
Code	A clean, current `/review` record (Step 11) that matches your code	Stops without shipping
Documents	A scan for secrets and personal data — no `/review` needed	Stops without shipping

> **`/ship` is a checkpoint — run it any time, not just at the end.** You don't have to wait until the product is finished. Run `/ship` after any step to save that step's work to the `dev` branch, as often as you like.

> **Note:** You'll know it worked when Claude's final message shows a deploy URL or completion confirmation.

> **Switch back to Opus 4.7 when `/ship` finishes.** Sonnet 5 is only for `/build`, `/review`, and `/ship` — return to **Opus 4.7** in the model picker before Step 13.

👁 Step 13 — Second opinion

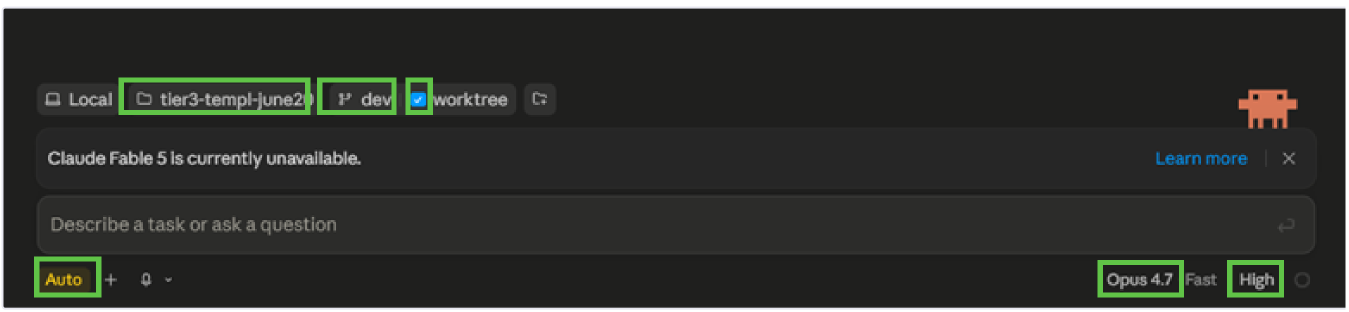
Claude Code · Opus 4.7

YOUR PART — *Open a new session and run `/second-opinion`.*

Start a fresh session on Opus 4.7. Open a **new Claude Code session** so the review runs with clean context — independent of the session that built the app — and set the model to **Opus 4.7** before running `/second-opinion`.

Configure the new session with the same settings as Step 2:

1. Working folder: your app folder from **Step 1**
2. Branch: **dev** (with **worktree** checked)
3. Mode: **auto**
4. Model: **Opus 4.7**, **high** effort



Same settings as Step 2: dev branch with worktree, Opus 4.7 at high effort, and auto mode.

In your new session, run **`/second-opinion`**. It takes a fresh look at the app you've built, reviewing and checking your work, then reports back what it finds so you can address anything before testing locally.

Note: Address anything it flags before moving on — a fresh-eyes pass catches issues the build session may have missed.

🧪 Step 14 — Test the app locally

Claude Code · Opus 4.7

> **YOUR PART —** *Run `/local-testing`.*

> **Stay in the same session as Step 13.** Run this in the `/second-opinion` session you just opened — no need to start another new one. Keep the model on **Opus 4.7**.

In **Claude Code**, run **`/local-testing`**. It starts the frontend and API services, sets up the local database, and launches the web app so you can view it locally. When it finishes, Claude gives you the localhost URLs — open the frontend URL in your browser to view the app.

Quick reference

Step	What you do	Model	What you get
1. Set up a new application	Install the project template + create app folder in PowerShell / Terminal	—	App folder ready at `C:\Users\[user_name]\claude_apps\[app_name]\`
2. Open your project in Claude Code	Launch Claude Desktop; point Claude Code at your app folder	Opus 4.7	Working folder set
3. Gather requirements	Run **`/requirements-gathering`** in Claude Code	Opus 4.7	Your requirements, written up
4. Create a blueprint	Run **`/design-foundation`**; review the sitemap and sign off on the selection	Opus 4.7	`full-design-blueprint.md` + `HANDOFF-design-brief.md` + `sitemap.html`
Setup — Design system *(if needed)*	Set up the McDermott Design System in Claude Design	—	Design system ready to use
5. Build the prototype	Drive Claude Design to build screens one at a time	—	Approved prototype
6. Share for review *(optional)*	Export the prototype and collect feedback	—	Refined prototype
7. Export the project archive	Download the prototype as a `.zip` from Claude Design	—	Project archive ready for reconciliation
8. Reconcile with prototype	Run **`/design-code-handoff`**; drag or upload the `.zip` into the chat	Opus 4.7	Reconciled blueprint (applied automatically)
9. Plan the build	Run **`/plan`** (or hand off to a developer)	Opus 4.7	Build plan
10. Build the product	Run **`/build`** in Claude Code	**Sonnet 5**	Implemented product
11. Review the code	Run **`/review`** in Claude Code	**Sonnet 5**	Clean review record that `/ship` requires

Step	What you do	Model	What you get
12. Ship the product	Run `/ship` in Claude Code	**Sonnet 5**	Deployed product
13. Second opinion	Run `/second-opinion` in a new Claude Code session	Opus 4.7	Independent review of your built app
14. Test the app locally	Run `/local-testing` in Claude Code	Opus 4.7	App running locally on your machine